



**UNIVERSIDADE FEDERAL DE RORAIMA
CAMPUS PARICARANA
CENTRO DE CIÊNCIAS TECNOLÓGICAS**



CURSO: Ciência da Computação

DISCIPLINA: COMP921 – Desenvolvimento de Jogos

Docente: Luciano Ferreira Silva

**Discente: Aurélio Arendartchuk Júnior, Gabriel Guilherme Figueiredo, Jonathan Soanno
Marchiori, Lucas Guilherme Pereira Santiago**

Nº Matrícula: 2022010070, 2024005507, 2024008821, 2022009329

DESIGN BIBLE DEATHTRAP DUNGEON

**BOA VISTA - RR
2026**

DESIGN BIBLE — DEATHTRAP DUNGEON

*Documento apresentado à disciplina de
COMP921 – Desenvolvimento de Jogos,
consolidando o Design Bible do projeto
Deathtrap Dungeon a partir da engenharia
reversa do código-fonte, dos dados de
balanceamento e da documentação original do
projeto.*

Boa Vista - RR
2026

Sumário

1. Introdução

1.1 Motivação

1.2 Objetivos

2. Design Bible

2.1 Roteiro

2.2 Game Design

2.3 Gameplay

2.4 Interface Gráfica

3. Detalhes de Desenvolvimento (Implementação dos Scripts)

3.1 Hierarquia de Classes de Combate

3.2 Padrões de Projeto

3.3 Sistema de Save/Load

3.4 Estrutura de Tilemap e Level Design

3.5 Limitações Técnicas Conhecidas

4. Avaliação

4.1 Metodologia de Avaliação

4.2 Resultados e Discussões

5. Conclusões

Referências

1. Introdução

1.1 Motivação

O desenvolvimento de jogos eletrônicos independentes envolve o desafio de equilibrar sistemas de combate, progressão de personagem e ambientação de forma coesa, especialmente em produções de pequena escala inspiradas no gênero Action-Adventure/Roguelike. O projeto Deathtrap Dungeon parte da necessidade de investigar como decisões de design — como janelas de animação de combate, sistemas de risco e recompensa e tabelas de balanceamento numérico — se traduzem em código funcional e em experiência de jogo. Documentar essas decisões em um Design Bible consolidado serve como exercício prático de análise de design e arquitetura de software para a disciplina de Desenvolvimento de Jogos.

1.2 Objetivos

O objetivo geral deste trabalho é consolidar, documentar e analisar as regras de design, as mecânicas de jogabilidade e a arquitetura técnica do projeto Deathtrap Dungeon, produzindo um Design Bible de referência para a equipe e para fins acadêmicos. De forma específica, pretende-se:

- Descrever os pilares de design que orientam o jogo, incluindo o combate tátil, o sistema de risco recompensado por fúria e a progressão legível de personagem e arma;
- Documentar o loop central de jogabilidade (gameplay loop) e os sistemas de combate, fúria e interação com objetos do cenário;
- Detalhar a arquitetura técnica do projeto e os padrões de projeto (design patterns) utilizados na implementação dos scripts em C#;
- Avaliar o escopo atual de implementação frente ao planejado, identificando limitações conhecidas e propondo sugestões de expansão.

2. Design Bible

2.1 Roteiro

Premissa. O jogador assume o papel de um aventureiro armado com uma lâmina que precisa sobreviver e progredir através de uma masmorra hostil, enfrentando monstros, armadilhas e um chefe final, enquanto acumula ouro (pesos) e experiência para se tornar mais forte. O núcleo da experiência gira em torno do combate ágil em tempo real, da progressão incremental de poder (nível e arma) e da exploração de ambientes de masmorra construídos

em Tilemap, com clara inspiração em jogos de ação 2D top-down do gênero Zelda-like/Roguelike.

Gênero, plataforma e público-alvo. O jogo se enquadra no gênero Action RPG / Hack-and-slash top-down, com subgênero de Roguelike de masmorra estruturado em fases lineares de dificuldade crescente. A plataforma-alvo é PC, desenvolvido em Unity, com câmera top-down 2D que segue o jogador. O público-alvo são jogadores de ação 2D casuais e hardcore que apreciam combate baseado em timing e progressão de personagem simples e direta.

Estrutura de fases (roteiro de progressão). O roteiro do jogador ao longo da masmorra segue a seguinte sequência de cenas:

- Begin — cena inicial/introdutória;
- Dungeon_0 — primeira área de masmorra;
- Dungeon_1 — segunda área de masmorra;
- Dungeon_Boss — arena do confronto final contra o chefe Boss0;
- End — cena de encerramento/vitória.

A transição entre cenas é feita por Portais, que salvam o progresso automaticamente antes de carregar a próxima cena de forma assíncrona, exibindo uma barra de progresso na interface.

2.2 Game Design

Os pilares abaixo orientam as decisões de sistema do jogo e devem guiar qualquer expansão futura do projeto:

Pilar	Descrição
Combate tátil e com peso	Cada golpe tem uma janela de animação calibrada (preparação, corte e recolhimento) para que o combate pareça físico e não instantâneo. O recuo (knockback) é parte central da sensação de impacto.
Risco recompensado por fúria	O jogador é incentivado a tomar dano com moderação: cada golpe recebido alimenta a barra de Fúria, que desbloqueia uma habilidade ofensiva poderosa.
Progressão legível	Nível, vida máxima, dano da arma e custo de upgrades seguem tabelas numéricas claras e crescentes, sem aleatoriedade nos ganhos.
Perigo previsível, porém implacável	Inimigos têm raios de detecção e perseguição bem definidos, mas o chefe final quebra essa regra ao enfurecer-se com pouca vida.
Mundo legível por meio do Tilemap	Camadas de piso, paredes, objetos de superfície e colisão são pensadas para clareza de leitura espacial imediata.

Protagonista

- **Vida inicial:** 10 pontos, aumentando +10 por nível ao subir de nível, sempre curando a vida atual para o novo máximo.
- **Movimento:** eixo horizontal com velocidade cheia; eixo vertical a 75% da velocidade horizontal, criando uma leve sensação de perspectiva apesar do top-down puro.
- **Tempo de imunidade após dano:** 0,75s, mais longo que o dos inimigos comuns, dando ao jogador uma folga extra para reagir.
- **Morte e reaparecimento:** ao chegar a 0 de vida, o jogador cai (rotação de 90°), a tela de morte é exibida e, após 6 segundos, ele reaparece com vida cheia na cena atual.

Progressão do Jogador

Dois eixos de progressão paralelos e independentes orientam o crescimento do personagem:

- **Nível de personagem (XP):** obtido ao derrotar inimigos, com valor de XP específico por tipo de inimigo. Subir de nível aumenta a vida máxima em +10 e cura o jogador por completo.
- **Nível de arma (ouro):** o ouro é coletado principalmente através de baús. No menu de personagem, o jogador pode gastar ouro para subir o nível da arma, aumentando dano e força de empurrão.

Esse design separa deliberadamente "ficar mais resistente" (XP → vida) de "causar mais dano" (ouro → arma), dando ao jogador duas fontes de progressão para perseguir simultaneamente.

Bestiário — Inimigos

Todos os inimigos padrão perseguem o jogador por meio de dois raios: um raio de ativação, que inicia a perseguição, e um raio de alcance máximo, a partir do qual o inimigo desiste e retorna à posição original. Um indicador visual de alerta aparece sobre o inimigo em perseguição, mudando de ícone quando a vida cai abaixo de 50%.

Arquétipo	ID técnico	Vida	XP	Dano	Perfil de combate
Rato/Batedor (baixo risco)	enemy_0	2	5	2	Frágil e fraco; inimigo introdutório, raio de ativação amplo.
Perseguidor Rápido	enemy_1	2	5	4	Vida frágil, dano dobrado, raio de ativação curto — pune aproximação descuidada.
Guarda Resistente	enemy_3	6	10	3	Tanque leve do início do jogo.

Arquétipo	ID técnico	Vida	XP	Dano	Perfil de combate
Brutamontes	enemy_4	12	10	5	Ameaça de médio porte, exige mais golpes para abater.
Investida Ágil	enemy_5	4	5	4	Vida baixa, alta velocidade, hit-and-run.
Guardião	enemy_6	10	25	5	Alta recompensa de XP, velocidade elevada.
Elite	enemy_7	18	20	10	Inimigo comum mais perigoso, equivalente a um mini-chefe.
Mímico (Monstro-Baú)	enemy_Chest	20	10	8	Disfarça-se de baú até entrar em perseguição; encontro único por sala.

Chefe: Boss0

O chefe possui 150 pontos de vida, 20 de dano de contato e recompensa 70 de XP; não reaparece, sendo um encontro único e definitivo. Duas bolas de fogo orbitam continuamente ao redor do chefe, funcionando como uma barreira de dano por contato que o jogador precisa aprender a contornar.

Ao cair para 20% ou menos de vida máxima, o chefe entra em uma fase de fúria permanente (enrage):

Parâmetro	Antes	Depois (Enrage)
Velocidade de órbita das bolas de fogo	2,5	4,0 (60% mais rápida)
Velocidade de movimento	0,6x	1,0x (velocidade total)
Raio de ativação	Base	2x o valor original
Raio de perseguição	Base	2x o valor original
Cor do sprite	Normal	Vermelho (sinalização visual)

Tabelas de Balanceamento

Progressão da arma (nível 0 a 6):

Nível da Arma	Dano	Força de Empurrão	Preço de Upgrade (ouro)
0	1	2.0	— (inicial)
1	2	2.2	10
2	4	2.5	50
3	6	3.0	100
4	9	3.3	180

Nível da Arma	Dano	Força de Empurrão	Preço de Upgrade (ouro)
5	12	3.6	300
6	16	4.0	500

Tanto o dano quanto o custo crescem de forma acelerada (não linear). O salto de custo entre os níveis 5 e 6 (+200) é o maior de toda a tabela, tornando o upgrade final um objetivo claro de fim de jogo.

Progressão de nível por experiência (XP):

Nível	XP necessário neste nível	XP acumulado total	Vida Máxima
1	0	0	10
2	10	10	20
3	20	30	30
4	30	60	40
5	40	100	50
6	50	150	60
7	60	210	70
8	70	280	80
9	80	360	90
10	90	450	100

O gradiente de XP cresce de forma linear e previsível (+10 de exigência a cada nível), em contraste com o crescimento acelerado do preço de upgrade da arma — subir de nível tende a ficar proporcionalmente mais fácil que evoluir a arma, reforçando a arma como o gargalo estratégico principal de progressão.

2.3 Gameplay

Loop Central de Gameplay

O ciclo de jogo pode ser descrito pela seguinte sequência: explorar a masmorra → encontrar inimigos, armadilhas e baús → combater (golpear, receber dano, acumular Fúria) → derrotar inimigos, ganhando XP e ocasionalmente ouro → subir de nível (vida máxima aumenta automaticamente) → gastar ouro para evoluir a arma → avançar para a próxima sala/fase através de portais → enfrentar o chefe final e vencer.

O ciclo de curto prazo (segundos) é golpear → recuar/desviar → repetir; o ciclo de médio prazo (minutos) é evoluir arma e nível; o ciclo de longo prazo (sessão completa) é atravessar a masmorra até o confronto com o chefe.

Sistema de Combate

- **Entrada e ataque básico:** clique esquerdo do mouse ou barra de espaço, com cooldown de 0,4s entre golpes, impedindo spam de ataques. A detecção de dano ocorre apenas durante a animação de swing.

A animação de corte é dividida em quatro fases, pensadas para simular um golpe real e garantir hit detection justa:

Fase	Ângulo	Velocidade	Collider	Função
P1 — Preparação	Arma sobe e recua	Moderada	Desativado	Antecipação do golpe (tell visual)
P2 — Corte principal	$90^{\circ} \rightarrow 0^{\circ}$	Rápida → muito rápida	Ativado	Fase de dano principal
P3 — Finalização	$0^{\circ} \rightarrow -20^{\circ}$	Foco em n° de frames	Ativado	Mantém a hitbox horizontal, de alcance maior
P4 — Recolhimento	Retorno à posição neutra	Rápida	Desativado	Reset para novo golpe

A animação de swing nunca deve ser menor que 0,3s, ou o jogador acerta o inimigo sem gerar um empurrão suficiente para se proteger do contra-ataque. Dano e força de empurrão escalam com o nível da arma (0 a 6), não com o nível do jogador. Se o jogador virar para o lado oposto durante o ataque, a animação é interrompida imediatamente e retorna ao estado idle, permitindo reposicionamento tático rápido.

Sistema de Fúria (Rage)

A barra de Fúria varia de 0 a 50. Toda vez que o jogador recebe dano (fora de uma habilidade de Fúria ativa), o valor do dano recebido é somado à barra. Ao atingir o máximo, o jogador pode ativar o estado de Fúria, com efeito visual próprio. Enquanto em Fúria, cada ataque básico dispara uma "Espada Flamejante", um projétil que causa dano à distância, transformando o combo corpo a corpo padrão em um ataque de alcance, por 4 segundos.

O sistema de Fúria recompensa jogadores que se arriscam em combate corpo a corpo, criando um contraponto de risco/recompensa ao padrão usual de evitar todo dano possível.

Objetos e Elementos Interativos

Objeto	Comportamento
Baú	Ao ser tocado, libera uma quantia de ouro (padrão 5), muda de sprite para "vazio" e exibe texto flutuante de recompensa. Só pode ser coletado uma vez.
Caixote	Objeto destrutível com vida própria; pode ser quebrado com a arma e é destruído ao chegar a 0 HP.
Porta	Muda visualmente para o sprite de "aberta" ao ser tocada pelo jogador.
Fonte de Cura	Cura o jogador em pequenos incrementos a cada 0,5s de contato, até esgotar uma reserva total, incentivando uso ponderado.
NPC de Diálogo	Exibe mensagens em sequência ao ser tocado, com cooldown de 4s entre falas e loop de múltiplas falas.
Armadilha	Pode ter um atraso de ativação configurável, permitindo arranjos de armadilhas dessincronizadas.
Portal	Salva o progresso e transiciona para outra cena ao ser tocado pelo jogador.

2.4 Interface Gráfica

- **CharacterHUD:** exibe em tempo real vida, experiência e fúria do jogador, além da tela de morte (game over).
- **CharacterMenu:** menu de personagem/inventário, incluindo a interface de upgrade de arma.
- **Textos flutuantes:** sistema reutilizável para feedback instantâneo de eventos (dano, cura, XP ganho, ouro coletado, "LEVEL UP!", falas de NPC), com cor, tamanho de fonte, posição, direção de movimento e duração configuráveis por chamada.
- **Tela de transição de cena (SCUI):** tela de carregamento com barra de progresso suavizada durante transições assíncronas de cena.

Arte e Estilo Visual

O jogo utiliza estilo de pixel art top-down, com tileset de 16×16 pixels e ambientação de masmorra de pedra clássica (fantasia sombria), com elementos ambientais como fontes de água e lava representados por tiles animados. Tanto o jogador quanto a arma trocam de sprite dinamicamente conforme nível/skin equipados, e inimigos exibem indicadores de estado (alerta / vida baixa) sobrepostos ao sprite base. O chefe muda para vermelho na fase de enrage, um padrão de sinalização por cor que pode ser expandido para outros inimigos de elite no futuro.

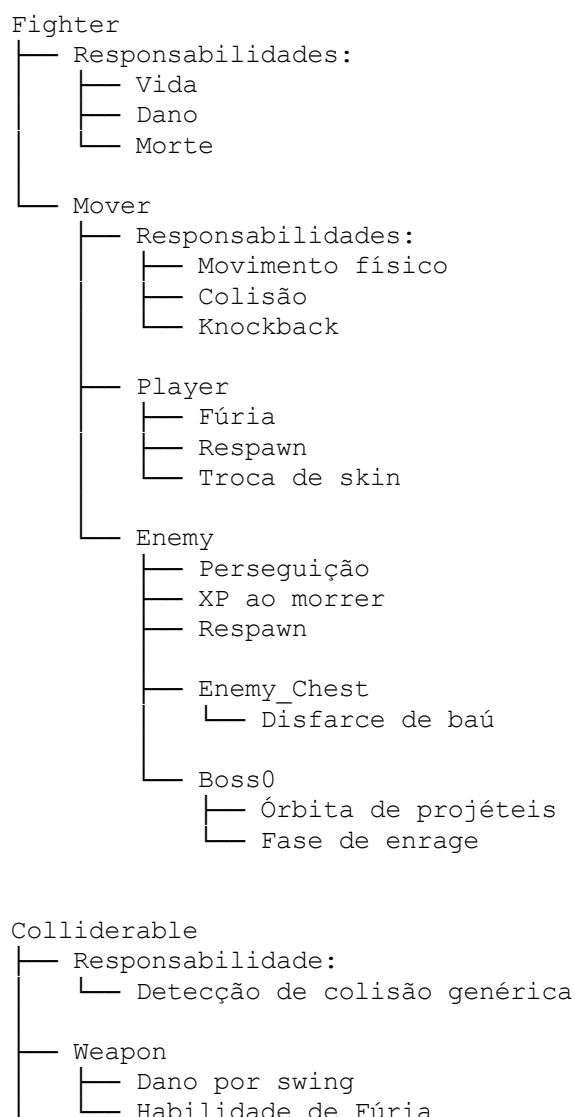
Áudio

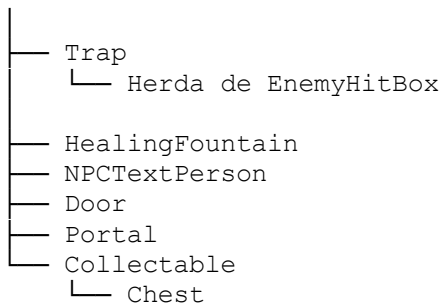
O áudio é gerenciado com arquitetura simples baseada em dicionário nome→clipe. A música de fundo utiliza uma única fonte de áudio dedicada, em loop, trocável por nome, enquanto efeitos sonoros são tocados em fontes extras alocadas dinamicamente conforme a demanda, reaproveitando fontes ociosas antes de criar novas — permitindo sobreposição de múltiplos efeitos simultâneos. O volume é controlado de forma unificada por um slider de interface, aplicado a todas as fontes de áudio ativas. A música de fundo utiliza o modo de carregamento Streaming, reduzindo significativamente o uso de memória em tempo de execução.

3. Detalhes de Desenvolvimento (Implementação dos Scripts)

3.1 Hierarquia de Classes de Combate

A arquitetura de combate segue uma hierarquia de herança organizada por responsabilidade:





3.2 Padrões de Projeto

- **Singleton:** um gerenciador central de estado global (ouro, XP, referências a Player/Weapon/UIManager/SaveManager) persiste entre cenas.
- **Herança por responsabilidade:** a classe base de combate cuida exclusivamente de vida e dano; a camada de movimento adiciona física por cima, permitindo que objetos que não se movem herdem apenas a lógica de vida e dano necessário.
- **Mensageria por SendMessage:** o dano é entregue aos alvos por meio de uma chamada de mensagem genérica, um padrão simples, embora com custo de desempenho maior que o uso de interfaces, adequado à escala atual do projeto.
- **Máquina de estados via Animator:** toda a lógica de fase de ataque (preparação, corte, finalização e recolhimento) é implementada no Animator da engine, não em código, mantendo o script da arma enxuto.

3.3 Sistema de Save/Load

O sistema de persistência utiliza serialização em JSON, salvando os dados em um arquivo local. São persistidos: o ouro acumulado, a experiência total, o nível da arma e o valor atual de Fúria. O jogo salva automaticamente ao atravessar um Portal. Ao carregar, se não existir arquivo de save, um novo save "zerado" é criado automaticamente, garantindo que o jogo nunca quebre por ausência de save. O nível do jogador não é salvo diretamente — é recalculado a partir da experiência total salva, usando a mesma tabela de XP, o que evita inconsistências entre XP e nível.

3.4 Estrutura de Tilemap e Level Design

Cada masmorra é organizada em camadas bem definidas: camada de piso (base visual do chão), camada de parede superior (pode ocultar o jogador ao passar por trás), camada de parede inferior (pode ocultar o jogador ao passar por baixo), camada de objetos de superfície (decoração e elementos interativos) e camada de colisão (define fisicamente onde jogador e

inimigos não podem passar). Tiles animados são usados para elementos como fontes de água e lava, dando vida ao cenário sem exigir objetos separados.

Duas regras de level design observadas na implementação: todos os objetos devem permanecer no mesmo plano $Z = 0$ para evitar falhas de detecção de colisão 2D; e a uniformidade de recursos de arte e tipo de tile é recomendada para reduzir chamadas de renderização (draw calls) — combinar dois estilos artísticos na mesma camada de parede é a principal fonte de sobrecarga de renderização identificada.

3.5 Limitações Técnicas Conhecidas

- **Condição de corrida na inicialização da arma:** o componente visual da arma precisa ser obtido antes que o gerenciador de jogo tente configurar o nível da arma, exigindo cuidado na ordem de inicialização dos scripts.
- **Resíduo de knockback no respawn:** corrigido ao zerar explicitamente a direção de empurrão quando o jogador ou inimigo está inativo, evitando que o personagem "voe" ao reaparecer.
- **Knockback atravessando paredes:** identificado como uma limitação conhecida, ainda sem correção definitiva registrada.

4. Avaliação

4.1 Metodologia de Avaliação

A avaliação deste projeto foi conduzida analisando três fontes principais: o código-fonte em C# de cada script do jogo, a planilha de dados de balanceamento e a documentação original do projeto. O processo consistiu em (i) mapear as classes e seus relacionamentos de herança para reconstruir a arquitetura técnica; (ii) tabular os valores numéricos de balanceamento (dano, vida, custos de upgrade) em uma planilha de dados; e (iii) confrontar o comportamento descrito no código de modo a identificar consistências, lacunas e limitações não resolvidas. Essa abordagem permite validar se as regras de design pretendidas foram efetivamente implementadas no código, sem depender exclusivamente de testes de jogo manuais.

4.2 Resultados e Discussões

A análise indicou que os seguintes sistemas estão implementados e funcionais:

- Sistema de combate completo (ataque, dano, empurrão, cooldown);

- Sistema de Fúria com projétil de longo alcance;
- Progressão de nível e de arma;
- Troca de cena entre fases, com salvamento automático;
- Tilemap completo com camadas de piso, paredes, colisão, objetos de superfície e tiles animados;
- Bestiário com 8 tipos de inimigos comuns e 1 chefe;
- Objetos interativos: baús, caixotes, portas, fontes de cura, NPCs de diálogo e armadilhas com atraso configurável;
- Sistema de save/load em JSON.

Do ponto de vista do balanceamento numérico, observa-se uma assimetria proposital entre os dois eixos de progressão: o custo de evolução da arma cresce de forma acelerada (não linear), enquanto o requisito de experiência para subir de nível cresce de forma linear. Essa escolha de design reforça a arma como o principal gargalo estratégico da progressão, dando ao jogador uma meta de médio/longo prazo mais desafiadora do que simplesmente acumular XP.

Quanto às limitações técnicas, a análise do código evidenciou uma condição de corrida na inicialização da arma (já contornada), um bug de resíduo de knockback no respawn (corrigido) e uma limitação ainda em aberto relacionada ao empurrão (knockback) atravessando paredes, o que indica uma área prioritária para correção em versões futuras. Na planilha de balanceamento, existe um slot de inimigo reservado (enemy_10) sem estatísticas preenchidas, sinalizando conteúdo planejado e ainda não implementado.

De forma geral, os resultados sugerem que a implementação está alinhada aos pilares de design propostos, especialmente o combate tátil e o risco recompensado por fúria, e que o escopo atual constitui uma base funcional e coerente para expansão futura do jogo.

5. Conclusões

Este Design Bible consolidou, a partir dos dados de balanceamento, as regras de design, os sistemas de jogabilidade e a arquitetura técnica do Deathtrap Dungeon. Verificou-se que os pilares de design definidos, como combate tátil, risco recompensado por fúria, progressão legível e mundo legível por Tilemap, encontram-se refletidos de forma consistente na implementação atual, e que os principais sistemas (combate, progressão, save/load, bestiário e chefe) já estão funcionais.

Possíveis expansões para trabalhos futuros, são:

- **IA de inimigos em repouso:** atualmente inimigos ficam parados até detectar o jogador; adicionar movimentação aleatória quando não estão perseguindo é uma melhoria sinalizada.
- **Combate entre inimigos:** possibilidade de inimigos se atacarem mutuamente (facções, criaturas hostis entre si), ainda não implementada.
- **Correção de knockback atravessando paredes:** bug conhecido, ainda sem solução definitiva.
- **Otimização adicional de memória:** aplicar técnicas de streaming já usadas na música de fundo a outros assets, além de considerar AssetBundles, redução de tamanho de textura e MipMaps.
- **Customização de fases:** o projeto já foi desenhado pensando em reuso, permitindo redesenhar o Tilemap, ajustar valores de balanceamento, reposicionar objetos de cena e reescrever falas de NPC sem alterar o código-base.

Conclui-se que o Deathtrap Dungeon apresenta uma base de design e implementação coerente e funcional, adequada como estudo de caso para a disciplina de Desenvolvimento de Jogos.

Referências

LOUMERIM. DeathtrapDungeon. Disponível em:

<https://github.com/LOUMERIM/DeathtrapDungeon/tree/main>.

UNITY TECHNOLOGIES. 2d-extras. Disponível em:

<https://github.com/Unity-Technologies/2d-extras>.

LitJson. Biblioteca de serialização JSON utilizada no sistema de Save/Load.